Software Development

JD522

FA2

Due: 06-09-2024

Thamsanqa Ngcobo

20231230

Date: 06-09-2024

Source code

```
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.List;
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.attribute.BasicFileAttributes;
import javax.swing.table.DefaultTableModel;
import java.sql.*;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
```

```
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTextField;

/**
 *
 * @author User 1
 */
public class TaskManagerForm extends javax.swing.JFrame {
  ArrayList<Task<Integer>> taskList = new ArrayList<>();
   private int taskIdCounter = 1;
   private Object tasks;
   /**
    * Creates new form TaskManagerForm
    */
   public TaskManagerForm() {
      initComponents();
   }

   /**
    * This method is called from within the constructor to initialize the form.
    * WARNING: Do NOT modify this code. The content of this method is always
    * regenerated by the Form Editor.
    */
   @SuppressWarnings("unchecked")
   // <editor-fold defaultstate="collapsed" desc="Generated Code">
   private void initComponents() {

      jCheckBox1 = new javax.swing.JCheckBox();
      jLabel1 = new javax.swing.JLabel();
      jLabel2 = new javax.swing.JLabel();
```

```
    jLabel3 = new javax.swing.JLabel();
    jLabel4 = new javax.swing.JLabel();
    jLabel5 = new javax.swing.JLabel();
    jBtnAddTask = new javax.swing.JButton();
    jBtnViewTask = new javax.swing.JButton();
    jTaskName = new javax.swing.JTextField();
    jTaskDescription = new javax.swing.JTextField();
    jComboCategory = new javax.swing.JComboBox<>();
    jScrollPane1 = new javax.swing.JScrollPane();
    jTableTasks = new javax.swing.JTable();
    jBtnSaveTasks = new javax.swing.JButton();
    jBtnReadTasks = new javax.swing.JButton();
    jButton1 = new javax.swing.JButton();
    jBtnExportCSV = new javax.swing.JButton();
    jLabel6 = new javax.swing.JLabel();
    jSearch = new javax.swing.JTextField();
    jBtnSearch = new javax.swing.JButton();
    jComboBoxShowCategory = new javax.swing.JComboBox<>();
    jLabel7 = new javax.swing.JLabel();

    jCheckBox1.setText("jCheckBox1");

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
    setBackground(java.awt.Color.lightGray);
    setCursor(new java.awt.Cursor(java.awt.Cursor.HAND_CURSOR));
    setForeground(javax.swing.UIManager.getDefaults().getColor("Actions.GreyInline"));
    setLocation(new java.awt.Point(300, 200));

    jLabel2.setFont(new java.awt.Font("Sitka Text", 1, 18)); // NOI18N
    jLabel2.setText("Task Manager");
```

```
    jLabel3.setFont(new java.awt.Font("Yu Gothic Medium", 0, 12)); // NOI18N
    jLabel3.setText("Task Name");

    jLabel4.setFont(new java.awt.Font("Yu Gothic Medium", 0, 12)); // NOI18N
    jLabel4.setText("Task Description");

    jLabel5.setFont(new java.awt.Font("Yu Gothic Medium", 0, 12)); // NOI18N
    jLabel5.setText("Category");

    jBtnAddTask.setFont(new java.awt.Font("Sylfaen", 0, 14)); // NOI18N
    jBtnAddTask.setText("Add task");
    jBtnAddTask.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnAddTaskActionPerformed(evt);
        }
    });

    jBtnViewTask.setFont(new java.awt.Font("Sylfaen", 0, 14)); // NOI18N
    jBtnViewTask.setText("View Tasks");
    jBtnViewTask.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnViewTaskActionPerformed(evt);
        }
    });

    jComboCategory.setFont(new java.awt.Font("Bell MT", 0, 14)); // NOI18N
    jComboCategory.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] {
"Personal", "School", "Work", "Health" }));

    jTableTasks.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {
            {null, null, null, null, null},
```

```
            {null, null, null, null, null},
            {null, null, null, null, null},
            {null, null, null, null, null}
        },
        new String [] {
            "ID", "Task Name", "Task Description", "Category", "Completion Status"
        }
    ));
    jScrollPane1.setViewportView(jTableTasks);

    jBtnSaveTasks.setText("Save Tasks");
    jBtnSaveTasks.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnSaveTasksActionPerformed(evt);
        }
    });

    jBtnReadTasks.setText("Read Tasks");
    jBtnReadTasks.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnReadTasksActionPerformed(evt);
        }
    });

    jButton1.setText("Properties");
    jButton1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jButton1ActionPerformed(evt);
        }
    });
```

```
    jBtnExportCSV.setText("Export To CSV");
    jBtnExportCSV.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnExportCSVActionPerformed(evt);
        }
    });

    jLabel6.setText("Search Task");

    jBtnSearch.setText("Search");
    jBtnSearch.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jBtnSearchActionPerformed(evt);
        }
    });

    jComboBoxShowCategory.setModel(new javax.swing.DefaultComboBoxModel<>(new
String[] { "Personal", "School", "Work", "Health" }));
    jComboBoxShowCategory.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            jComboBoxShowCategoryActionPerformed(evt);
        }
    });

    jLabel7.setText("Show Category");

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(156, 156, 156)
```

```
        .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 37,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

    .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addComponent(jLabel2, javax.swing.GroupLayout.PREFERRED_SIZE, 270,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

    .addGroup(layout.createSequentialGroup()

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

            .addGroup(layout.createSequentialGroup()

                .addGap(38, 38, 38)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

                .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

                    .addComponent(jBtnSaveTasks, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

                    .addComponent(jBtnAddTask, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

                    .addGap(94, 94, 94)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

                    .addComponent(jBtnViewTask, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

                    .addComponent(jBtnReadTasks, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)))

                .addGroup(layout.createSequentialGroup()

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING, false)

                    .addComponent(jLabel6, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
```

```
                    .addComponent(jLabel4, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                    .addComponent(jLabel3, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                    .addComponent(jLabel5, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.PREFERRED_SIZE, 67, javax.swing.GroupLayout.PREFERRED_SIZE)

                    .addGroup(layout.createSequentialGroup()

                        .addGap(13, 13, 13)

                        .addComponent(jLabel7, javax.swing.GroupLayout.PREFERRED_SIZE, 84,
javax.swing.GroupLayout.PREFERRED_SIZE)))

                    .addGap(99, 99, 99)


.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

                    .addGroup(layout.createSequentialGroup()

                        .addGap(83, 83, 83)

                        .addComponent(jSearch, javax.swing.GroupLayout.PREFERRED_SIZE, 143,
javax.swing.GroupLayout.PREFERRED_SIZE))


.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)

                        .addComponent(jComboBoxShowCategory,
javax.swing.GroupLayout.PREFERRED_SIZE, 141, javax.swing.GroupLayout.PREFERRED_SIZE)


.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

                        .addComponent(jTaskName)

                        .addComponent(jTaskDescription)

                        .addComponent(jComboCategory, 0, 141, Short.MAX_VALUE))))))

            .addGap(31, 31, 31))

            .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
layout.createSequentialGroup()

            .addContainerGap()

            .addComponent(jBtnSearch)

            .addGap(78, 78, 78)))

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 527,
javax.swing.GroupLayout.PREFERRED_SIZE)

            .addGroup(layout.createSequentialGroup()

                .addGap(6, 6, 6)

                .addComponent(jButton1, javax.swing.GroupLayout.PREFERRED_SIZE, 95,
javax.swing.GroupLayout.PREFERRED_SIZE)

                .addGap(48, 48, 48)

                .addComponent(jBtnExportCSV)))

        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

    );

    layout.setVerticalGroup(

        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addGroup(layout.createSequentialGroup()

            .addGap(34, 34, 34)

            .addComponent(jLabel1)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

            .addComponent(jLabel2)

            .addGap(47, 47, 47)

            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

                .addGroup(layout.createSequentialGroup()


.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

                    .addComponent(jLabel3, javax.swing.GroupLayout.PREFERRED_SIZE, 24,
javax.swing.GroupLayout.PREFERRED_SIZE)

                    .addComponent(jTaskName, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

                .addGap(18, 18, 18)


.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

                    .addComponent(jLabel4, javax.swing.GroupLayout.PREFERRED_SIZE, 46,
javax.swing.GroupLayout.PREFERRED_SIZE)

                    .addComponent(jTaskDescription, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))

                .addGap(18, 18, 18)
```

```
.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jLabel5, javax.swing.GroupLayout.PREFERRED_SIZE, 25, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(jComboCategory, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 28, Short.MAX_VALUE)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jComboBoxShowCategory, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addComponent(jLabel7))
            .addGap(18, 18, 18)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(jLabel6)
                .addComponent(jSearch, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(jBtnSearch)
            .addGap(22, 22, 22)

.addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
                .addComponent(jBtnAddTask, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(jBtnViewTask, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)))
        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 0, Short.MAX_VALUE))
    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jBtnSaveTasks)
        .addComponent(jBtnReadTasks)
        .addComponent(jButton1)
```

```
            .addComponent(jBtnExportCSV))
        .addGap(13, 13, 13))
    );

    pack();
  }// </editor-fold>

  private void jBtnAddTaskActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
     // Gather user input from text fields and combo box
    String taskName = jTaskName.getText();
    String description = jTaskDescription.getText();
    String category = jComboCategory.getSelectedItem().toString();

    // Validate user input (Ensure taskName and description are not empty)
    if (taskName.isEmpty() || description.isEmpty()) {
      // Display an error message to the user if either field is empty
      JOptionPane.showMessageDialog(this, "Task Name and Description are required.", "Error",
JOptionPane.ERROR_MESSAGE);
      return; // Exit the method if validation fails
    }

    // Generate a unique task ID using the counter
    int taskId = taskIdCounter++;

    // Create a Task object and add it to the ArrayList
    Task<Integer> newTask = new Task<>(taskId, taskName, description, false, category);
    taskList.add(newTask);

    // Database connection parameters
    String url = "jdbc:sqlite:C:/Users/Thami/TaskManagerDB.db/"; // Connection string
```

```
    String sql = "INSERT INTO TaskManager (task_name, Description, Completion_status, Category) VALUES (?, ?, ?, ?)"; // SQLite query

    try (Connection connection = DriverManager.getConnection(url);
         PreparedStatement preparedStatement = connection.prepareStatement(sql)) {

        // Set parameters for the SQL statement/query
        preparedStatement.setString(1, taskName);
        preparedStatement.setString(2, description);
        preparedStatement.setBoolean(3, false); // Initial completion status is set to false
        preparedStatement.setString(4, category);

        // Execute the SQL statement/query
        preparedStatement.executeUpdate();

        // Clear input fields
        jTaskName.setText("");
        jTaskDescription.setText("");
        jComboCategory.setSelectedIndex(0);

        // Display a success message to the user
        JOptionPane.showMessageDialog(this, "Task added successfully.", "Success", JOptionPane.INFORMATION_MESSAGE);

    } catch (SQLException ex) {
        // Handle/Validate database-related exceptions (e.g., connection failure, SQL errors)
        ex.printStackTrace();
        // Display an error message to the user
        JOptionPane.showMessageDialog(this, "Error adding task: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

```
    private void jBtnViewTaskActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String url = "jdbc:sqlite:C:/Users/Thami/TaskManagerDB.db/"; // Connection string
        String sql = "SELECT * FROM TaskManager"; // SQLite query

        try (Connection connection = DriverManager.getConnection(url);
             Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {

            // Create a DefaultTableModel to hold the task data
            DefaultTableModel tableModel = new DefaultTableModel();
            tableModel.addColumn("ID");
            tableModel.addColumn("Task Name");
            tableModel.addColumn("Description");
            tableModel.addColumn("Completion Status");
            tableModel.addColumn("Category");

            // Populate the table model with task data from the database
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String taskName = resultSet.getString("task_name");
                String description = resultSet.getString("description");
                boolean completionStatus = resultSet.getBoolean("completion_status");
                String category = resultSet.getString("category");

                // Add a row to the table model for each task
                tableModel.addRow(new Object[]{id, taskName, description, completionStatus,
category});
            }

            // Set the table model for the JTable
```

```
        jTableTasks.setModel(tableModel);

    } catch (SQLException ex) {
        // Handle database-related exceptions (e.g., connection failure, SQL errors)
        ex.printStackTrace();
        // Display an error message to the user
        JOptionPane.showMessageDialog(this, "Error viewing task: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

  private void jBtnSaveTasksActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    // File chooser to select the location where the file will be saved
  JFileChooser fileChooser = new JFileChooser();
  fileChooser.setDialogTitle("Save Tasks");

  int userSelection = fileChooser.showSaveDialog(this);

  if (userSelection == JFileChooser.APPROVE_OPTION) {
    File fileToSave = fileChooser.getSelectedFile();

    try (FileOutputStream fos = new FileOutputStream(fileToSave);
         BufferedOutputStream bos = new BufferedOutputStream(fos);
         PrintWriter writer = new PrintWriter(bos)) {
        Iterable<Task> tasks = null;

        // Loop through your tasks and write them to the file
        for (Task task : tasks) {
            writer.println(task.toString()); // Modify toString() in Task to format task details
        }
```

```
        writer.flush();
        JOptionPane.showMessageDialog(this, "Tasks saved successfully!");
    } catch (IOException ex) {
        JOptionPane.showMessageDialog(this, "Error saving tasks: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
    }
  }
  }

  private void jBtnReadTasksActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    // Define the file path
  String filePath = "tasks.txt";

  // Clear the existing task list
  taskList.clear();

  try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
    String line;

    // Read each line from the file
    while ((line = reader.readLine()) != null) {
        String[] parts = line.split("\\|"); // Adjust the delimiter as necessary

        // Ensure the parts array has the expected length
        if (parts.length == 5) {
            // Create a new Task object and add it to the list
            Task<Integer> task = new Task<>(
                Integer.parseInt(parts[0]),   // ID
                parts[1],                     // Task Name
                parts[2],                     // Description
                Boolean.parseBoolean(parts[3]),// Completion Status
```

```
            parts[4]            // Category
        );
        taskList.add(task);
    }
}

    // Optionally update the JTable to reflect the new tasks
    updateTaskTable();

} catch (IOException | NumberFormatException e) {
    // Handle exceptions
    e.printStackTrace();
    JOptionPane.showMessageDialog(this, "Error reading tasks: " + e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
}
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
        Path path = Paths.get("C:/Users/Thami/TaskManagerDB.db");

try {
    // Get file attributes
    BasicFileAttributes attrs = Files.readAttributes(path, BasicFileAttributes.class);

    // Display file properties
    String message = String.format("File Size: %d bytes\nCreation Time: %s\nLast Modified
Time: %s",
        attrs.size(),
        attrs.creationTime(),
        attrs.lastModifiedTime());
```

```
        JOptionPane.showMessageDialog(this, message, "File Properties",
JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException ex) {
        // Handle the exception
        ex.printStackTrace();
        JOptionPane.showMessageDialog(this, "Error retrieving file properties: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
    }

    }

    private void jBtnExportCSVActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
         Path path = Paths.get("C:/Users/Thami/Tasks.csv");

    try {
        // Prepare the header for CSV
        String header = "ID,Task Name,Description,Completion Status,Category\n";

        // Prepare the rows
        StringBuilder sb = new StringBuilder();
        sb.append(header);

        for (Task<Integer> task : taskList) {
            sb.append(task.getId()).append(',')
              .append(task.getTaskName()).append(',')
              .append(task.getDescription()).append(',')
              .append(task.isCompletionStatus()).append(',')
              .append(task.getCategory()).append('\n');
        }

        // Write data to CSV file
```

```
    Files.write(path, sb.toString().getBytes(StandardCharsets.UTF_8));

    // Notify user
    JOptionPane.showMessageDialog(this, "Tasks exported successfully to CSV.", "Export Success", JOptionPane.INFORMATION_MESSAGE);
  } catch (IOException ex) {
    // Handle the exception
    ex.printStackTrace();
    JOptionPane.showMessageDialog(this, "Error exporting tasks to CSV: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
  }
}

private void jBtnSearchActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String searchTerm = jSearch.getText().toLowerCase();
  DefaultTableModel tableModel = (DefaultTableModel) jTableTasks.getModel();

  // Clear existing rows
  tableModel.setRowCount(0);

  for (Task<Integer> task : taskList) {
    if (task.getTaskName().toLowerCase().contains(searchTerm) ||
        task.getDescription().toLowerCase().contains(searchTerm) ||
        task.getCategory().toLowerCase().contains(searchTerm)) {

        tableModel.addRow(new Object[]{
            task.getId(),
            task.getTaskName(),
            task.getDescription(),
            task.isCompletionStatus(),
            task.getCategory()
```

```
        });
    }
}
}

private void jComboBoxShowCategoryActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

}

    HashMap<String, ArrayList<Task<Integer>>> categorizedTasks = new HashMap<>();
        private void categorizeTasks() {
categorizedTasks.clear(); // Clear previous categorization

for (Task<Integer> task : taskList) {
    String category = task.getCategory();
    if (!categorizedTasks.containsKey(category)) {
        categorizedTasks.put(category, new ArrayList<>());
    }
    categorizedTasks.get(category).add(task);
}
}
public class TaskManagerGUI extends JFrame {

// Declare the components at the top level
private JPanel panelTaskEntry;
private JTextField txtTaskName;
private JTextField txtDescription;
private JComboBox<String> cmbCategory;
private JButton btnAddTask;
```

```
    // Constructor to set up the GUI
    public TaskManagerGUI() {
        // Initialize components
        panelTaskEntry = new TaskEntryPanel();
        this.add(panelTaskEntry);
        // Set frame properties
        this.setTitle("Task Manager");
        this.setSize(600, 400);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setVisible(true);
    }

    // Inner class for Task Entry Panel
    private class TaskEntryPanel extends JPanel {
        public TaskEntryPanel() {
            // Set layout
            this.setLayout(new GridLayout(4, 2, 10, 10));

            // Initialize components
            JLabel lblTaskName = new JLabel("Task Name:");
            JLabel lblDescription = new JLabel("Description:");
            JLabel lblCategory = new JLabel("Category:");
            txtTaskName = new JTextField();
            txtDescription = new JTextField();
            cmbCategory = new JComboBox<>(new String[] {"Work", "Personal", "Urgent"});
            btnAddTask = new JButton("Add Task");

            // Add action listener to button
            btnAddTask.addActionListener(e -> addTask());

            // Add components to panel
```

```
        this.add(lblTaskName);
        this.add(txtTaskName);
        this.add(lblDescription);
        this.add(txtDescription);
        this.add(lblCategory);
        this.add(cmbCategory);
        this.add(btnAddTask);
    }

    private void addTask() {
        String taskName = txtTaskName.getText();
        String description = txtDescription.getText();
        String category = (String) cmbCategory.getSelectedItem();
        // Logic to add task
        System.out.println("Task added: " + taskName + ", " + description + ", " + category);
    }
}

  // Inner class for Category Management
  private class CategoryPanel extends JPanel {
    private JComboBox<String> cmbCategories;
    private JButton btnShowCategory;

    public CategoryPanel() {
        // Set layout
        this.setLayout(new FlowLayout());

        // Initialize components
        cmbCategories = new JComboBox<>(new String[] {"Work", "Personal", "Urgent"});
        btnShowCategory = new JButton("Show Tasks");
```

```
        // Add action listener to button
        btnShowCategory.addActionListener(e -> showTasksByCategory());

        // Add components to panel
        this.add(cmbCategories);
        this.add(btnShowCategory);
    }

    private void showTasksByCategory() {
        String selectedCategory = (String) cmbCategories.getSelectedItem();
        // Logic to display tasks by selected category
        System.out.println("Showing tasks for category: " + selectedCategory);
    }
  }



}

  /**
   * @param args the command line arguments
   */
  public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
```

```
            break;
        }
    }
    } catch (ClassNotFoundException ex) {
java.util.logging.Logger.getLogger(TaskManagerForm.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (InstantiationException ex) {
java.util.logging.Logger.getLogger(TaskManagerForm.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (IllegalAccessException ex) {
java.util.logging.Logger.getLogger(TaskManagerForm.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
java.util.logging.Logger.getLogger(TaskManagerForm.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new TaskManagerForm().setVisible(true);
        }
    });
}

  // Variables declaration - do not modify
  private javax.swing.JButton jBtnAddTask;
  private javax.swing.JButton jBtnExportCSV;
  private javax.swing.JButton jBtnReadTasks;
```

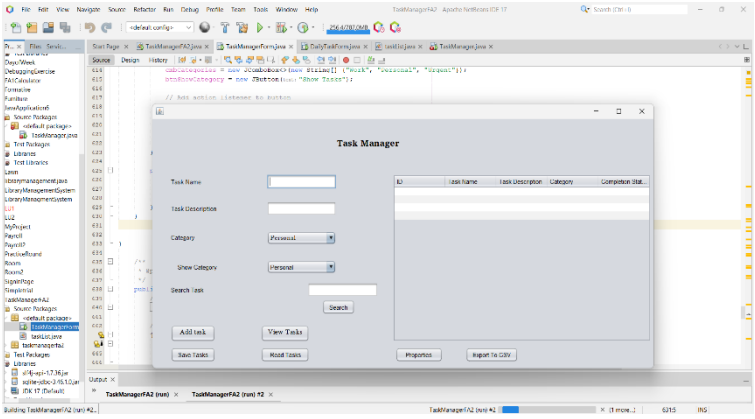
```
    private javax.swing.JButton jBtnSaveTasks;
    private javax.swing.JButton jBtnSearch;
    private javax.swing.JButton jBtnViewTask;
    private javax.swing.JButton jButton1;
    private javax.swing.JCheckBox jCheckBox1;
    private javax.swing.JComboBox<String> jComboBoxShowCategory;
    private javax.swing.JComboBox<String> jComboCategory;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JLabel jLabel5;
    private javax.swing.JLabel jLabel6;
    private javax.swing.JLabel jLabel7;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextField jSearch;
    private javax.swing.JTable jTableTasks;
    private javax.swing.JTextField jTaskDescription;
    private javax.swing.JTextField jTaskName;
    // End of variables declaration

    private void updateTaskTable() {
        throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
    }
}
class Task<T> {
    private T id;
    private String taskName;
    private String description;
    private boolean completionStatus;
    private String category;
```

```
    public Task(T id, String taskName, String description, boolean completionStatus, String category) {
        this.id = id;
        this.taskName = taskName;
        this.description = description;
        this.completionStatus = completionStatus;
        this.category = category;
    }

    Task(String name, String description, String dueDate) {
        throw new UnsupportedOperationException("Not supported yet."); // Generated from nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
    }

    // Getters and setters for each field
    public T getId() {
        return id;
    }

    public void setId(T id) {
        this.id = id;
    }

    public String getTaskName() {
        return taskName;
    }

    public void setTaskName(String taskName) {
        this.taskName = taskName;
    }
```

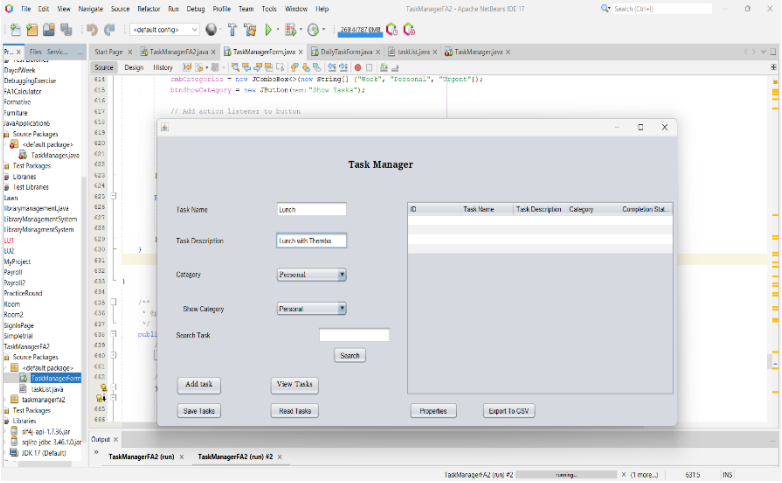
```
    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public boolean isCompletionStatus() {
        return completionStatus;
    }

    public void setCompletionStatus(boolean completionStatus) {
        this.completionStatus = completionStatus;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    @Override
    public String toString() {
        return "Task{" +
                "id=" + id +
                ", taskName='" + taskName + '\'' +
                ", description='" + description + '\'' +
                ", completionStatus=" + completionStatus +
```

```
            ", category='" + category + '\'' +
            '}';
    }
}
```

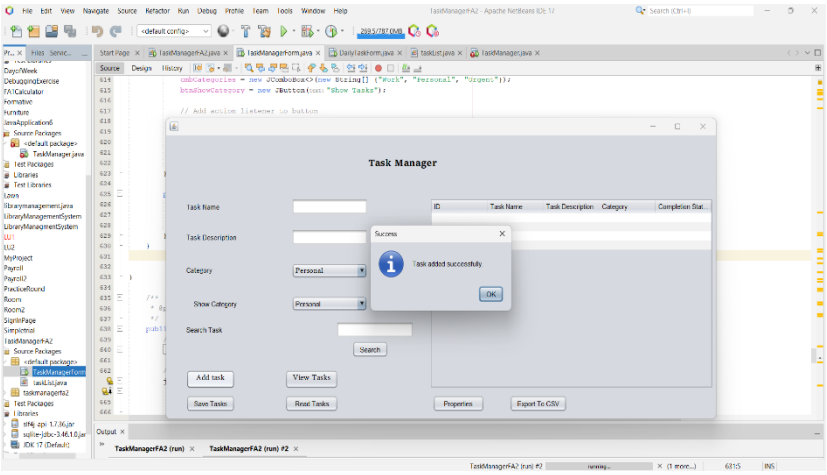
Screenshots



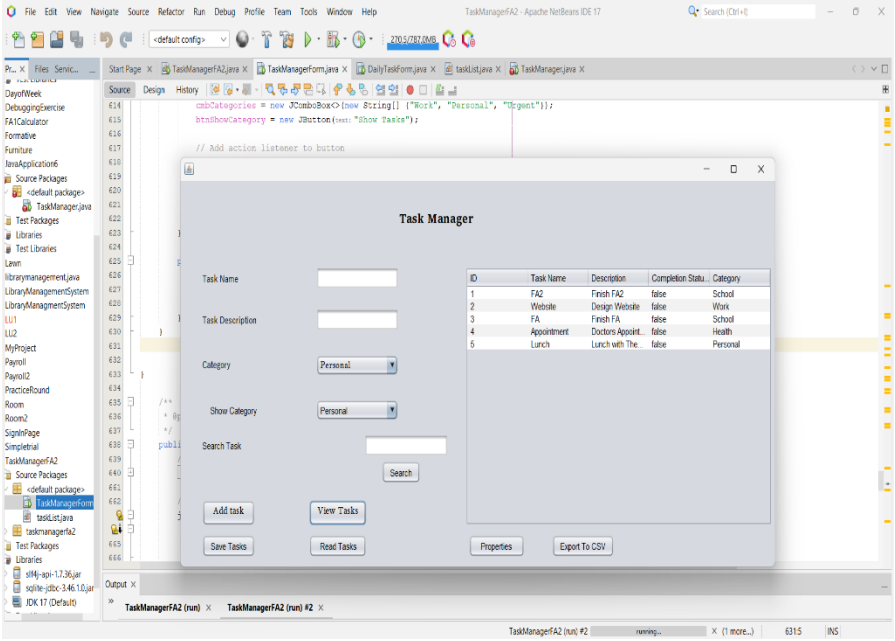
GUI



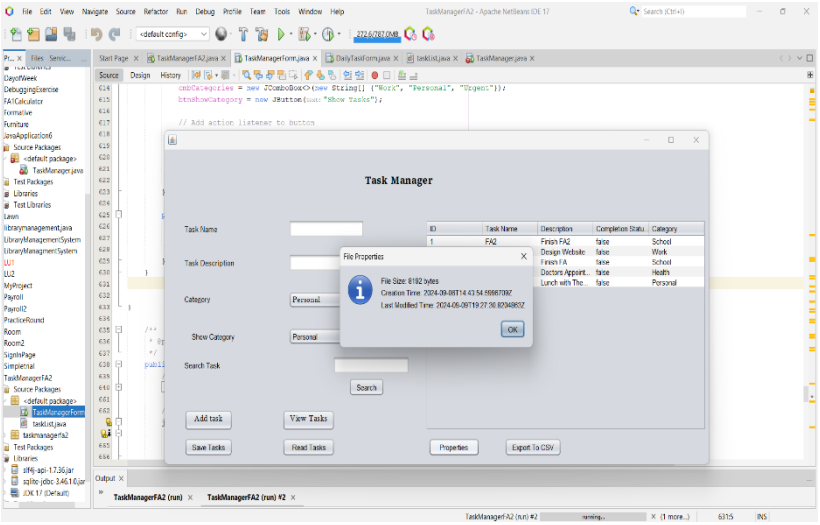
Adding a Task



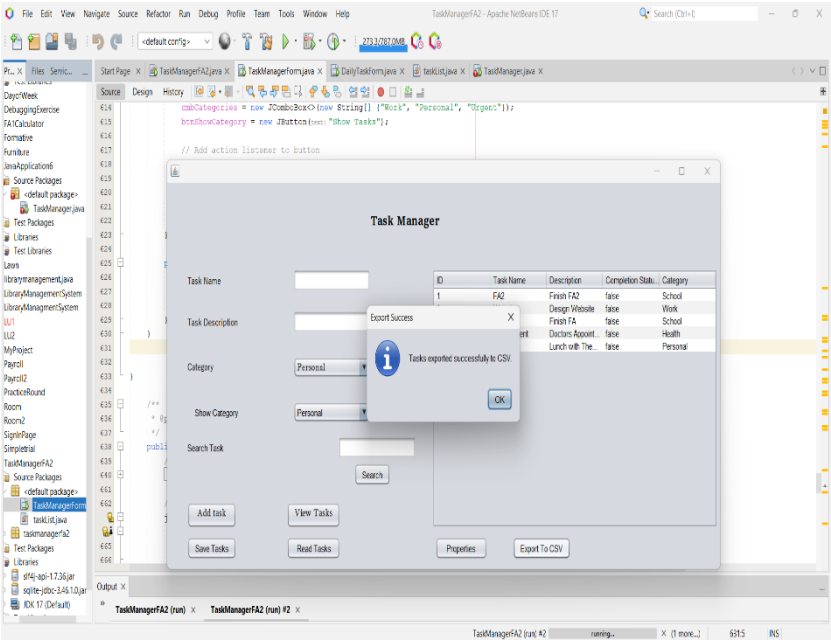
Task added



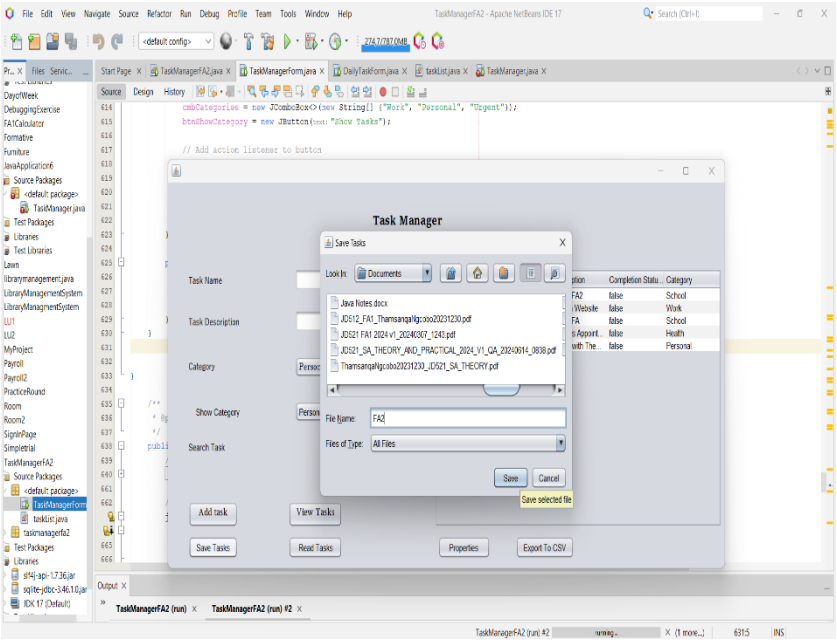
View Task button, shows already added tasks and newly added task.



Properties Button shows the properties of the project



Export CSV button



Save tasks button